CSS Notes

Cascading Style Sheets

Student Study Notes | Syntax + Explanation + Examples

Contents

1. Introduction to CSS
2. CSS Syntax
3. Types of CSS (How to add CSS)
4. Colors
5. Backgrounds
6. Borders
7. Margin and Padding
8. Height and Width
9. Gradients
10. Shadows
11. Selectors
12. Transformations (2D and 3D)
13. Typography (Fonts and Text)
14. The CSS Box Model
15. Outline
16. Transitions
17. Navigation Bar
18. CSS Combinators
19. Pseudo-classes and Pseudo-elements
20. Tooltips
21. Images
22. Buttons
23. Animations
24. User Interface Properties
25. Box Sizing
26. Filters
27. Responsive Web Design

1. Introduction to CSS

CSS (Cascading Style Sheets) describes how HTML elements look: colours, fonts, spacing, layout and animations. HTML gives the structure; CSS gives the style.

- **Cascading** means that when many rules apply to one element, the browser decides which wins using order, specificity and importance.
- CSS separates content (HTML) from presentation, so one stylesheet can style many pages.
- Current version: CSS3 (made of many modules).
- Files are saved with the extension **.css**.

```
p {
  color: blue;
  font-size: 18px;
}
```

2. CSS Syntax

```
selector {
  property: value;
  property: value;
}

h1 {
  color: red;
  text-align: center;
}
```

Part	Meaning
Selector	Chooses the HTML element(s) to style (h1).
Declaration block	Everything inside { }.
Declaration	One property : value pair ending with a semicolon.
Property	What to change (color, margin).
Value	The setting for the property (red, 10px).

- **Comments**: /* this is a comment */
- **Grouping selectors**: h1, h2, p { color: green; }
- **!important** forces a rule to win: color: red !important;

3. Types of CSS (How to add CSS)

Type	Where it is written
Inline	In the style attribute of one element.
Internal	In a <style> tag inside <head>.
External	In a separate .css file linked with <link>.

```
<!-- 1. Inline CSS -->
<p style="color:red; font-size:20px;">Inline text</p>
```

```
<!-- 2. Internal CSS -->
<head>
  <style>
    p { color: blue; }
  </style>
</head>

<!-- 3. External CSS (best practice) -->
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

- **Priority** (highest to lowest): Inline > Internal/External (the one written last wins) > Browser default.
- External CSS is best: one file controls the whole website and is cached by the browser.

4. Colors

Format	Example
Name	color: tomato; (140 named colours)
HEX	color: #ff6347; (#RRGGBB)
RGB	color: rgb(255, 99, 71);
RGBA	color: rgba(255, 99, 71, 0.5); (a = opacity 0 to 1)
HSL	color: hsl(9, 100%, 64%); (hue, saturation, lightness)
HSLA	color: hsla(9, 100%, 64%, 0.5);

```
h1 { color: #333333; background-color: rgb(255, 235, 59); }
p  { color: hsl(200, 80%, 40%); }
div { background-color: rgba(0, 0, 0, 0.3); }   /* semi-transparent */
```

Note: color changes text colour; background-color changes the background colour.

5. Backgrounds

Property	Use
background-color	Solid background colour.
background-image	url("img.jpg") sets an image.
background-repeat	repeat | no-repeat | repeat-x | repeat-y
background-position	center, top, left, 50% 50% ...
background-size	cover | contain | 100px 50px
background-attachment	scroll | fixed (parallax effect)
background	Shorthand for all of the above.

```
body {
  background-color: #f0f0f0;
  background-image: url("bg.jpg");
  background-repeat: no-repeat;
  background-position: center;
```

```
  background-size: cover;
  background-attachment: fixed;
}

/* Shorthand */
div { background: #fff url("bg.png") no-repeat center / cover; }
```

6. Borders

Property	Use
border-width	thin | medium | thick | 2px
border-style	solid, dashed, dotted, double, groove, ridge, none
border-color	Any colour value.
border	Shorthand: width style color
border-radius	Rounded corners (50% makes a circle).
border-top / right / bottom / left	Border on one side only.

```
div {
  border: 2px solid #333;
  border-radius: 10px;
}
p {
  border-bottom: 3px dashed red;
  border-style: solid dotted;   /* top-bottom  left-right */
}
img { border-radius: 50%; }     /* circle image */
```

7. Margin and Padding

Margin is the space **outside** the border (between elements). **Padding** is the space **inside** the border (between content and border).

```
div {
  margin: 20px;                 /* all 4 sides */
  padding: 10px 20px;           /* top-bottom 10, left-right 20 */
}
p {
  margin: 10px 15px 20px 25px;  /* top right bottom left (clockwise) */
  padding-left: 30px;
  margin-top: 0;
}
.center { margin: 0 auto; width: 300px; }   /* center a block horizontally */
```

Shorthand values	Applied to
1 value	All four sides.
2 values	top & bottom | left & right
3 values	top | left & right | bottom
4 values	top | right | bottom | left

> **Note:** Vertical margins of neighbouring blocks can **collapse**: the larger margin is used.

8. Height and Width

```
div {
  width: 300px;
  height: 150px;
  max-width: 100%;      /* never wider than parent */
  min-height: 100px;    /* at least this tall */
  overflow: auto;       /* scrollbar if content is larger */
}
img { width: 100%; height: auto; }
```

- Units: **px** (pixel), **%** (of parent), **em/rem** (font based), **vw/vh** (viewport width/height), **auto**.
- Block elements default to width:auto (full width). Inline elements ignore width and height.
- **overflow** values: visible | hidden | scroll | auto.

9. Gradients

Gradients show a smooth transition between two or more colours. They are set using background-image.

```
/* Linear: direction + colours */
.a { background: linear-gradient(to right, red, yellow); }
.b { background: linear-gradient(45deg, #ff6b6b, #4ecdc4, #1a535c); }

/* Radial: from centre outward */
.c { background: radial-gradient(circle, white, blue); }

/* Conic: around a centre point */
.d { background: conic-gradient(red, yellow, green, blue, red); }

/* Colour stops */
.e { background: linear-gradient(red 0%, yellow 50%, green 100%); }
```

> **Note:** Use repeating-linear-gradient() / repeating-radial-gradient() for repeating stripes.

10. Shadows

Property	Syntax
box-shadow	x-offset y-offset blur spread color [inset]
text-shadow	x-offset y-offset blur color

```
.card {
  box-shadow: 5px 5px 10px rgba(0,0,0,0.4);
}
.card2 {
  box-shadow: 0 4px 8px gray, 0 0 0 3px blue;   /* multiple shadows */
}
.inner { box-shadow: inset 0 0 10px black; }     /* inside shadow */

h1 { text-shadow: 2px 2px 4px #999; }
```

11. Selectors

A **selector** chooses which HTML elements a rule applies to.

Selector	Meaning
`*`	All elements.
`p`	Element (type) selector: all <p>.
`.intro`	Class selector: elements with class="intro".
`#main`	ID selector: the element with id="main".
`p.intro`	Only <p> elements with class intro.
`h1, h2`	Group: both h1 and h2.
`a[target]`	Attribute selector: links having target attribute.
`a[href^="https"]`	href starts with https.
`a[href$=".pdf"]`	href ends with .pdf.
`a[href*="google"]`	href contains google.

```
* { margin:0; padding:0; }
.intro { font-weight:bold; }
#main  { width:80%; }
input[type="text"] { border:1px solid gray; }
```

Specificity (which rule wins)

Inline style (1000) > ID (100) > Class, attribute, pseudo-class (10) > Element (1). If equal, the rule written last wins.

12. Transformations (2D and 3D)

The **transform** property moves, rotates, scales or skews an element.

Function	Effect
`translate(x, y)`	Move element.
`rotate(deg)`	Rotate clockwise (negative = anticlockwise).
`scale(x, y)`	Enlarge or shrink (1 = same size).
`skew(xdeg, ydeg)`	Tilt element.
`rotateX / rotateY / rotateZ`	3D rotation.
`matrix()`	All 2D transforms in one function.

```
.a { transform: translate(50px, 20px); }
.b { transform: rotate(30deg); }
.c { transform: scale(1.5); }
.d { transform: skew(20deg, 10deg); }
.e { transform: rotate(45deg) scale(1.2); }   /* combine */

.f { transform-origin: top left; }            /* change pivot point */
```

```
/* 3D */
.scene { perspective: 600px; }
.cube  { transform: rotateY(60deg); transform-style: preserve-3d; }
```

13. Typography (Fonts and Text)

Font properties

Property	Use
font-family	Font list; last is a generic fallback (serif, sans-serif, monospace).
font-size	px, em, rem, %.
font-weight	normal | bold | 100-900
font-style	normal | italic | oblique
font-variant	small-caps
font	Shorthand: style weight size/line-height family

Text properties

Property	Use
color	Text colour.
text-align	left | right | center | justify
text-decoration	none | underline | line-through | overline
text-transform	uppercase | lowercase | capitalize
text-indent	Indent of first line.
letter-spacing / word-spacing	Space between letters / words.
line-height	Height of each line (1.5 is good).
white-space	nowrap | pre | normal
text-overflow	ellipsis (needs overflow:hidden and nowrap).

```
/* Google Font */
@import url("https://fonts.googleapis.com/css2?family=Roboto&display=swap");

body {
  font-family: "Roboto", Arial, sans-serif;
  font-size: 16px;
  line-height: 1.6;
}
h1 { text-transform: uppercase; letter-spacing: 2px; text-align: center; }
a  { text-decoration: none; }
```

14. The CSS Box Model

Every HTML element is a rectangular **box** made of four layers (from inside to outside):

Layer	Description
Content	The text or image itself (width and height).
Padding	Transparent space around the content.
Border	Line around the padding.
Margin	Transparent space outside the border.

```
div {
  width: 300px;
  padding: 20px;
  border: 5px solid black;
  margin: 10px;
}
/* Total width = 300 + 20+20 + 5+5 = 350px (content-box)
   Space used incl. margin = 350 + 10+10 = 370px           */
```

Note: Use box-sizing: border-box (see Box Sizing) to make width include padding and border.

15. Outline

An **outline** is a line drawn **outside the border**. Unlike border, it does **not take space** and does not change the element size.

```
button:focus {
  outline: 3px dashed blue;
  outline-offset: 5px;      /* gap between border and outline */
}
div { outline-width: 2px; outline-style: solid; outline-color: red; }
```

- Properties: outline-width, outline-style, outline-color, outline-offset, and the shorthand outline.
- Do not remove focus outlines (outline:none) without giving another visible focus style: it harms keyboard users.

16. Transitions

Transitions make a property change smoothly over time instead of instantly (e.g., on hover).

Property	Use
transition-property	Which property (width, color, all).
transition-duration	Time: 0.5s, 300ms.
transition-timing-function	ease | linear | ease-in | ease-out | ease-in-out
transition-delay	Wait before starting.
transition	Shorthand: property duration timing delay

```
.btn {
  background: blue;
  width: 100px;
  transition: background 0.5s ease, width 0.5s ease 0.1s;
}
.btn:hover {
```

```
  background: red;
  width: 150px;
}
```

17. Navigation Bar

A navigation bar is a list of links styled as a menu. Build it with an unordered list.

Horizontal navbar

```
<ul class="nav">
  <li><a href="#">Home</a></li>
  <li><a href="#">About</a></li>
  <li><a href="#">Contact</a></li>
</ul>

<style>
  .nav { list-style:none; margin:0; padding:0;
         background:#333; display:flex; }
  .nav a { display:block; color:white; padding:14px 20px;
           text-decoration:none; }
  .nav a:hover { background:#555; }
</style>
```

Vertical navbar and sticky navbar

```
.vnav { list-style:none; width:200px; background:#f1f1f1; padding:0; }
.vnav a { display:block; padding:10px; color:#000; }

.sticky { position:sticky; top:0; }      /* stays at top while scrolling */
.fixed  { position:fixed; top:0; width:100%; }
```

18. CSS Combinators

A **combinator** explains the relationship between selectors.

Combinator	Meaning and example
`Descendant (space)`	div p : all <p> inside a (any depth).
`Child (>)`	div > p : <p> that are direct children of .
`Adjacent sibling (+)`	h1 + p : the first <p> immediately after h1.
`General sibling (~)`	h1 ~ p : all <p> siblings after h1.

```
div p   { color: red; }
div > p { color: blue; }
h1 + p  { font-weight: bold; }
h1 ~ p  { color: gray; }
```

19. Pseudo-classes and Pseudo-elements

Pseudo-classes (single colon :)

Style an element in a **special state**.

Pseudo-class	Meaning
:hover	Mouse is over the element.
:active	Element is being clicked.
:focus	Element has focus (input selected).
:visited / :link	Visited / unvisited link.
:first-child / :last-child	First / last child of its parent.
:nth-child(n)	Nth child: odd, even, 3, 2n+1.
:not(selector)	Everything except selector.
:checked / :disabled	Form states.

```
a:link    { color: blue; }
a:visited { color: purple; }
a:hover   { color: red; }       /* order: link, visited, hover, active (LoVe HAte)
*/
a:active  { color: orange; }
tr:nth-child(even) { background: #eee; }
input:focus { border-color: green; }
```

Pseudo-elements (double colon ::)

Pseudo-element	Meaning
::before / ::after	Insert content before / after element (needs content property).
::first-line	First line of text.
::first-letter	First letter (drop cap).
::selection	Text selected by the user.
::placeholder	Placeholder text of inputs.

```
p::first-letter { font-size: 200%; color: red; }
h2::before { content: "\2605 "; }     /* star symbol */
::selection { background: yellow; }
```

20. Tooltips

A **tooltip** is a small text box shown when the user hovers over an element. It can be made using only CSS.

```
<div class="tip">Hover me
  <span class="tiptext">Tooltip text</span>
</div>

<style>
  .tip { position: relative; display: inline-block; cursor: pointer; }
  .tip .tiptext {
    visibility: hidden;
    width: 140px; background: #333; color: #fff;
    text-align: center; padding: 6px; border-radius: 6px;
    position: absolute; bottom: 125%; left: 50%;
    margin-left: -70px;
    opacity: 0; transition: opacity 0.3s;
```

```
  }
  .tip:hover .tiptext { visibility: visible; opacity: 1; }
</style>
```

Note: Simple tooltips can also be made with the HTML title attribute.

21. Images

```
img { max-width: 100%; height: auto; }              /* responsive image */
.round  { border-radius: 50%; }                       /* circle */
.thumb  { border: 1px solid #ddd; padding: 5px; border-radius: 4px; }
.cover  { width: 300px; height: 200px; object-fit: cover; }   /* crop to fit */
.center { display: block; margin: 0 auto; }
.fade   { opacity: 0.5; }
.fade:hover { opacity: 1; }
```

object-fit value	Effect
`fill`	Stretch to fill (may distort).
`contain`	Whole image visible, keeps ratio.
`cover`	Fills the box, keeps ratio, crops extra.
`none / scale-down`	No resizing / smaller of none and contain.

- Use **object-position** to choose the visible area.
- Image sprites use background-position to show part of one big image.

22. Buttons

```
.btn {
  background-color: #04aa6d;
  color: white;
  border: none;
  padding: 12px 28px;
  font-size: 16px;
  border-radius: 6px;
  cursor: pointer;
  transition: 0.3s;
}
.btn:hover    { background-color: #038a57; }
.btn:active   { transform: scale(0.97); }
.btn:disabled { opacity: 0.5; cursor: not-allowed; }
.btn-outline  { background: transparent; border: 2px solid #04aa6d; color:
#04aa6d; }
.btn-round    { border-radius: 50px; }
.btn-block    { display: block; width: 100%; }
```

- Style <a> links as buttons using the same class with display:inline-block.
- Group buttons using display:flex and gap.

23. Animations

An **animation** changes styles gradually between keyframes. Define it with **@keyframes**, then apply with the **animation** property.

```
@keyframes slide {
  0%   { left: 0;     background: red; }
  50%  { left: 200px; background: yellow; }
  100% { left: 0;     background: red; }
}
.box {
  position: relative;
  width: 100px; height: 100px;
  animation: slide 4s ease-in-out 1s infinite alternate;
}
```

Property	Use
animation-name	Name of @keyframes.
animation-duration	Length of one cycle.
animation-timing-function	ease, linear, …
animation-delay	Start delay.
animation-iteration-count	Number of times, or infinite.
animation-direction	normal | reverse | alternate
animation-fill-mode	forwards keeps the final state.
animation-play-state	running | paused

24. User Interface Properties

Property	Use
resize	both | horizontal | vertical | none : user can resize element (needs overflow other than visible).
outline-offset	Space between outline and border.
cursor	pointer, default, text, move, not-allowed, wait, grab …
user-select	none stops text selection.
pointer-events	none makes element ignore mouse.
appearance	Remove default native styling of controls.
caret-color	Colour of the text cursor.
accent-color	Colour of checkboxes/radios.

```
textarea { resize: vertical; }
.link   { cursor: pointer; }
.noselect { user-select: none; }
input[type="checkbox"] { accent-color: green; }
```

25. Box Sizing

By default (**content-box**) width and height apply only to the content, so padding and border make the box larger. With **border-box**, width and height include padding and border, which makes layouts easier.

```
/* Recommended reset for all elements */
*, *::before, *::after {
```

```
  box-sizing: border-box;
}

.a { box-sizing: content-box; width:200px; padding:20px; border:5px solid; } /*
total 250px */
.b { box-sizing: border-box;  width:200px; padding:20px; border:5px solid; } /*
total 200px */
```

26. Filters

The **filter** property applies visual effects to images and elements. **backdrop-filter** blurs what is behind an element.

Function	Effect
blur(5px)	Blur.
brightness(150%)	Lighter / darker.
contrast(200%)	Contrast.
grayscale(100%)	Black and white.
sepia(100%)	Old photo effect.
hue-rotate(90deg)	Shift colours.
invert(100%)	Invert colours.
opacity(50%)	Transparency.
drop-shadow(4px 4px 5px gray)	Shadow that follows shape.

```
img { filter: grayscale(100%); transition: 0.4s; }
img:hover { filter: grayscale(0) brightness(110%); }
.glass { backdrop-filter: blur(8px); background: rgba(255,255,255,0.3); }
.multi { filter: contrast(120%) sepia(40%); }
```

27. Responsive Web Design

Responsive design makes a site look good on phones, tablets and desktops using flexible layouts, flexible images and **media queries**.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">

/* Mobile-first approach */
.container { width: 100%; padding: 10px; }
.col { width: 100%; }

@media (min-width: 600px) {          /* tablets */
  .col { width: 50%; float: left; }
}
@media (min-width: 992px) {          /* desktops */
  .col { width: 33.33%; }
}
@media (orientation: landscape) { body { font-size: 18px; } }
@media print { nav { display: none; } }
```

Flexbox and Grid (modern layout)

```
.row  { display:flex; flex-wrap:wrap; gap:10px; justify-content:space-between;
align-items:center; }
.item { flex: 1 1 200px; }              /* grow shrink basis */

.grid { display:grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
gap:15px; }
```

Unit	Based on
`%`	Parent size.
`em`	Parent font-size.
`rem`	Root (html) font-size.
`vw / vh`	1% of viewport width / height.